servers over SSH with the appropriate syntax.

Using Rsync with some useful options

1.*--exclude and --include*

As the flag name, we can use these options to specify which files to include and which to exclude during the sync.

The following command demonstrates this. It specifies the inclusion of all files with a *.py* extension and the exclusion of all files with a *.js* extension.

```
$: rsync -avze ssh --include '*.py' --exclude '*.js'
test/ root@xxx.xxx.xxx.xxx:/home/
root@xxx.xxx.xxx.xxx password:
sending incremental file list
./

sent 157 bytes  received 28 bytes  33.64 bytes/sec
total size is 25,049  speedup is 135.40
```

2. *--delete*

If a file or directory is not located at the source, but resides at the destination, you may want to delete it at the target while syncing to keep the latest copy of the backup. To do this, you have to use the *--delete* option, as follows:

```
$: rsync -avze ssh --delete test/ root@xxx.xxx.xxx.xxx:/
home/
root@xxx.xxx.xxx.xxx's password:
sending incremental file list
deleting flask_app_bk.py
./
sent 122 bytes  received 253 bytes  68.18 bytes/sec
total size is 13,298  speedup is 35.46
```

Deleting source files after a successful transfer

The option *--remove-source-files* is used to delete source files after a successful transfer, as shown below:

```
$: rsync -avze ssh --remove-source-files test/ root@xxx.
xxx.xxx.xxx:/home/
root@xxx.xxx.xxx.xxx's password:
sending incremental file list
./
flask_app_bk.py

sent 2,853 bytes  received 46 bytes  527.09 bytes/sec
total size is 11,751  speedup is 4.05

$: cd test/
$: ls
$:
```

Displaying the progress while transferring data with Rsync

To display the progress of the data being transferred from one machine to another, we can use the *--progress* option. The following code displays the files and the time remaining to complete the transfer:

```
$: rsync -avze ssh --progress test/ root@xxx.xxx.xxx.xxx:/
home/
root@xxx.xxx.xxx.xxx's password:
sending incremental file list
./
binjs.js
176 100%      0.00kB/s     0:00:00 (xfr#1, to-chk=3/5)
hook.js
3,124 100%    2.98MB/s     0:00:00 (xfr#2, to-chk=2/5)
test.js
3,876 100%    3.70MB/s     0:00:00 (xfr#3, to-chk=1/5)
test_bk.zip
3,729,290 100%  487.08kB/s  0:00:07 (xfr#4, to-chk=0/5)

sent 2,824,460 bytes  received 95 bytes  77,385.07 bytes/sec
total size is 3,736,466  speedup is 1.32
```

Automating Rsync backups with Cron

The Cron utility is used to automate command execution at a specific time or after fixed intervals. We can combine Rsync and Cron to automate backup tasks.

To edit the Cron table file for the user you are logged in as, run the following command:

```
$ crontab -e
```

Now press *i* to enter the *insert* mode and edit the Cron table file. Cron uses the following syntax: minute of the hour, hour of the day, day of the month, month of the year, day of the week, and command.

Now, if we want to sync the directory named *scripts* to */tmp/backup* every midnight, then the following line in the Cron table will do it with Rsync:

```
0 00 * * * rsync -avh --delete /home/user/programs/ /tmp/
backups
```

The first *0* specifies the minute of the hour, and *00* specifies the time as midnight. Since we want this command to run daily, we will leave the rest of the fields with asterisks and then write the Rsync command to be executed. END

By: Chetan Tarale

The author is an independent programmer and is interested in application security, reverse engineering and penetration testing. He also loves to explore GNU/Linux and emerging open source technologies.